ESTÁCIO S.A
Turma 3003

RODRIGO DE SOUSA BARBOSA
Matrícula: 202403618168
MANOEL DE SOUSA SANTOS
Matrícula: 202402239521
LUCAS ROBERT LOPES SILVA
Matrícula: 201902231597
ALAN FREITAS MIRANDA
Matrícula: 202404621073
GABRIEL HENRIQUE MACIEL LEITE
Matrícula: 202403628988
JORGE IGOR DOS SANTOS PINTO
Matrícula: 202309399989

SISTEMA DE CONTROLE DE VENDAS EM C

Cadastro de produtos, registro de vendas e relatórios

Relatório técnico - Programação de Software Básico em C
Professora: Suzane Alfaia Dias

2026

SUMÁRIO

1 INTRODUÇÃO 2

2 OBJETIVOS 2

2.1 Objetivo geral 2

2.2 Objetivos específicos 2

3 REQUISITOS DO SISTEMA 3

3.1 Funcionalidades e requisitos técnicos 3

3.2 Entradas, saídas e regras 3

4 DESENVOLVIMENTO 4

4.1 Arquitetura e decisões 4

4.2 Estruturas de dados e persistência 4

4.3 Ponteiros, alocação e transações 4

4.4 Fluxograma geral 5

4.5 Funções principais 5

5 TESTES E RESULTADOS 6

5.1 Testes da API e do banco 6

5.2 Interface e memória 7

5.3 Evidências da interface 8

6 MANUAL DE UTILIZAÇÃO 10

6.1 Compilar e executar 10

6.2 Operação diária 10

6.3 Testes, cópia de segurança e documentos 11

7 CONCLUSÃO 11

8 REFERÊNCIAS 12

9 APÊNDICE - ROTEIRO DE APRESENTAÇÃO 13

1 INTRODUÇÃO

Pequenos estabelecimentos precisam registrar produtos e vendas de maneira consistente. Anotações dispersas dificultam o acompanhamento do estoque, aumentam o risco de erros de cálculo e tornam a apuração dos resultados mais trabalhosa. O problema escolhido consiste em reunir essas informações em uma aplicação local, com interface gráfica e recuperação dos dados após o encerramento do programa.

O sistema Venda. permite cadastrar produtos e seus preços, registrar vendas, calcular automaticamente o total de cada operação e gerar relatórios do total vendido. A proposta também inclui estoque mínimo, descontos, formas de pagamento e cancelamento com devolução dos produtos, para demonstrar regras relacionadas ao tema.

O desenvolvimento segue os requisitos do documento Projeto Prático — Desenvolvimento de Software em C, fornecido com a atividade. O núcleo da solução é escrito em C11, dividido em arquivos .c e .h. A interface gráfica é apresentada no navegador; o servidor HTTP, as validações, os cálculos, as transações e o acesso ao SQLite são executados em C.

O público considerado é o responsável por uma pequena loja, em uso local, além dos integrantes da equipe durante a demonstração acadêmica. Não é necessário acesso à internet para executar o sistema depois de obter os arquivos do projeto.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver uma aplicação em C com interface gráfica e persistência em arquivo para organizar produtos, registrar vendas e apresentar o total vendido com integridade de estoque e precisão monetária.

2.2 Objetivos específicos

Implementar cadastro, listagem, pesquisa, atualização e exclusão de produtos; validar identificadores e entradas; aplicar regras de estoque e desconto; preservar o histórico de cada venda; demonstrar estruturas, ponteiros e memória dinâmica; recuperar informações após reiniciar; executar testes com dados válidos e inválidos; e documentar a compilação e o uso.

3 REQUISITOS DO SISTEMA

3.1 Funcionalidades e requisitos técnicos

Código	Requisito do PDF	Atendimento
R01	Interface gráfica com menu, retorno e encerramento	Quatro seções, modais, Ctrl+C e docker compose stop.
R02	Struct com identificador e cinco campos relevantes	Product, Sale e SaleItem em include/app.h.
R03	Cadastrar, listar, pesquisar, atualizar e excluir	CRUD de produtos e filtros no catálogo.
R04	Regras específicas do tema	Venda, baixa de estoque, cálculo, cancelamento e relatório.
R05	Condicionais e repetição	Validações, laços de itens e laço do servidor.
R06	Funções e modularização	main, http, api, database e utils.
R07	Ponteiros e memória dinâmica	calloc de itens e malloc/calloc de buffers; verificação e free.
R08	Persistência em arquivos	Arquivo SQLite, transações e reabertura.
R09	Validação de entradas	Campos vazios, tipos, limites, estoque, datas e SKU único.
R10	Arquivos .c e .h organizados	Fontes próprios separados de bibliotecas incorporadas.

3.2 Entradas, saídas e regras

As entradas são código/SKU, nome, categoria, preço, estoque, estoque mínimo, situação e observações dos produtos. Na venda, o usuário informa os produtos, as quantidades, o cliente opcional, a forma de pagamento, o desconto e observações. As saídas incluem catálogo, histórico, detalhes, comprovante interno, indicadores, gráfico diário e exportação CSV.

O preço unitário deve ser positivo. Quantidades são inteiras; estoque negativo e venda acima do saldo são proibidos. O desconto não pode superar o subtotal. O código/SKU é único, com comparação sem distinção de maiúsculas ASCII. O identificador interno é gerado pelo SQLite. Produtos com histórico são desativados, em vez de excluídos, para preservar as referências.

O total de uma venda é a soma de quantidade × preço unitário menos o desconto. Todos os cálculos monetários do backend usam inteiros de 64 bits em centavos. Uma venda pode ter até 200 produtos distintos, preço unitário de até R$ 1.000.000,00 e subtotal de até R$ 90.000.000.000,00. Estoques são limitados a 1.000.000 de unidades. Campos de texto têm limites em bytes UTF-8.

4 DESENVOLVIMENTO

4.1 Arquitetura e decisões

O servidor foi implementado em C com sockets POSIX e atende somente o endereço local 127.0.0.1. Ele entrega tanto os arquivos da interface quanto os endpoints JSON, mantendo uma única origem de acesso. O navegador usa HTML, CSS e JavaScript sem framework. O JavaScript apresenta os dados e calcula apenas uma prévia do carrinho; os valores definitivos são recalculados em C, a partir dos preços do banco.

O SQLite foi escolhido por armazenar o banco em um arquivo e oferecer transações e chaves estrangeiras sem exigir um servidor de banco separado. A biblioteca cJSON converte os corpos JSON recebidos e as respostas. Os códigos dessas duas bibliotecas estão incluídos em vendor, com identificação e licença. Os arquivos da aplicação são compilados com -Wall, -Wextra, -Wpedantic e -Werror.

Módulo	Responsabilidade
main.c	Parâmetros, abertura do banco e inicialização.
http.c	Sockets, limites de requisição, arquivos gráficos e encerramento.
api.c	CRUD, validação, venda, cancelamento, consultas e relatórios.
database.c	Esquema, abertura, transações e dados opcionais de exemplo.
utils.c	Conversões, validação de texto/número/data e respostas JSON.
app.h	Structs, constantes e contratos entre módulos.

4.2 Estruturas de dados e persistência

Estrutura	Campos principais
Product	id, sku, name, category, price_cents, stock, min_stock, active, notes.
Sale	id, customer, payment_method, subtotal_cents, discount_cents, total_cents, notes, items, item_count.
SaleItem	product_id, sku, name, quantity, unit_price_cents e total_cents.

As tabelas products, sales e sale_items representam os registros persistentes. Uma venda possui vários itens, e cada item referencia o produto original. O nome, o código e o preço unitário são copiados para o item da venda. Assim, editar o catálogo não modifica o histórico. O banco usa restrições CHECK, código único e chaves estrangeiras.

4.3 Ponteiros, alocação e transações

A quantidade de itens de uma venda só é conhecida em execução. A função create_sale aloca um vetor de SaleItem com calloc, verifica o retorno e libera o vetor com free tanto no

sucesso quanto nos caminhos de falha. Ponteiros também são usados nas estruturas de requisição, conexão SQLite, comandos preparados e buffers de comunicação.

```
sale.items = calloc(sale.item_count, sizeof(*sale.items));
if (!sale.items)
    return error_response(500, "Memória insuficiente...");
/* O vetor é liberado ao concluir ou desfazer a operação. */
free(sale.items);
```

Antes de ler e gravar os itens, a venda inicia BEGIN IMMEDIATE. O programa valida o estoque atual, calcula os valores, insere a venda e seus itens e atualiza os estoques. COMMIT confirma o conjunto. Qualquer falha leva a ROLLBACK, evitando venda parcial. O cancelamento também é transacional e só devolve estoque uma vez. O preço informado pelo cliente não é usado como fonte do valor definitivo.

4.4 Fluxograma geral

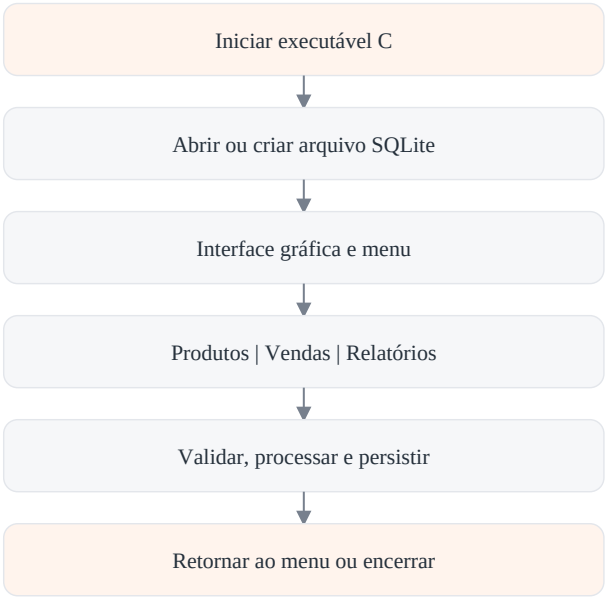


Figura 1 — Fluxo geral do sistema. Fonte: elaboração para este projeto.

4.5 Funções principais

db_open prepara o arquivo e o esquema. api_route seleciona a operação. save_product valida e persiste o produto; get_products permite pesquisa; delete_product protege o histórico. create_sale calcula e grava a venda; cancel_sale faz a reposição; get_report agrega os resultados. http_run mantém o ciclo de atendimento e http_stop encerra o servidor após responder à interface.

5 TESTES E RESULTADOS

Os testes foram executados contra o binário C real, por HTTP, utilizando arquivos SQLite isolados em tests/.runtime. A recuperação foi comprovada pelo encerramento e pela inicialização de outro processo com o mesmo arquivo. A integridade foi verificada com PRAGMA integrity_check e PRAGMA foreign_key_check. Os resultados abaixo foram extraídos dos registros JSON da execução, e não apenas previstos.

5.1 Testes da API e do banco

Teste	Entrada ou ação	Resultado esperado	Obtido
01 — Banco vazio	Consultar listas e relatório sem registros	Listas vazias e total zero	APROVADO
02 — CRUD	Criar, buscar, alterar e excluir produto	Dados atualizados; exclusão confirmada	APROVADO
03 — Duplicidade	Repetir código no cadastro/edição	HTTP 409; código único	APROVADO
04 — Produto inválido	Vazio, preço zero e estoque inválido	HTTP 400; nenhuma gravação	APROVADO
05 — Venda e histórico	2 × 24,90 + 39,90 – 9,70	Total 80,00; estoque reduzido; histórico fixo	APROVADO
06 — Atomicidade	Segundo item sem estoque	HTTP 409; nenhum saldo alterado	APROVADO
07 — Venda inválida	Vazia, repetida, inativa ou desconto alto	Recusar operação inválida	APROVADO
08 — Cancelamento	Cancelar e tentar cancelar novamente	Reposição única; total ajustado	APROVADO
09 — Relatórios	Somar vendas e filtrar datas	Somas exatas; datas inválidas recusadas	APROVADO
10 — Inexistentes	Consultar ID ausente e rota inválida	HTTP 404/405; arquivos protegidos	APROVADO
11 — Reinício	Encerrar e reabrir o mesmo banco	Recuperar produtos, vendas e totais	APROVADO
12 — HTTP/JSON	Origem externa e corpo inválido	Recusar com 403/400/415	APROVADO
13 — Centavos	3 × 0,10 – 0,01	Total exato de 0,29	APROVADO
14 — Restrições SQL	Tentar estoque negativo e SKU repetido	SQLite rejeita alterações	APROVADO
15 — Interface estática	Solicitar HTML, JS, CSS e SVG	HTTP 200 e tipos corretos	APROVADO
16 — Limite de reposição	Cancelamento ultrapassaria estoque máximo	Recusar sem alterar a venda	APROVADO

Teste	Entrada ou ação	Resultado esperado	Obtido
17 — Exemplos	Iniciar duas vezes com --demo	8 produtos e 6 vendas, sem duplicar	APROVADO
18 — Encerramento	Acionar /api/shutdown e reiniciar	Saída normal e dados preservados	APROVADO

Resultado registrado: 18 testes aprovados, 0 falhas e 0 erros. Execução: 2026-11-05T19:20:12.059218+00:00. Arquivo: artifacts/test-results.json.

5.2 Interface e memória

Fluxo no Chrome	Resultado obtido
Dashboard com dados reais do SQLite	APROVADO
Cadastro pelo formulário atualiza o catálogo	APROVADO
Código duplicado exibe erro visível e mantém formulário	APROVADO
Edição do preço e exclusão confirmada pelo navegador	APROVADO
Carrinho calcula venda e backend baixa estoque	APROVADO
Detalhe e cancelamento devolvem estoque e preservam histórico	APROVADO
Relatório exporta CSV com total correto e gera impressão PDF	APROVADO
Layout em celular sem transbordamento horizontal	APROVADO
Nenhum erro JavaScript durante os fluxos	APROVADO

Os mesmos 18 testes da API também passaram com AddressSanitizer e UndefinedBehaviorSanitizer, com detecção de vazamentos habilitada nos módulos próprios da aplicação. As bibliotecas incorporadas SQLite e cJSON foram ligadas sem instrumentação. Isso verifica o conjunto de fluxos executados e não equivale a provar ausência de qualquer falha em entradas possíveis. Evidência: artifacts/sanitizer-test-results.json.

5.3 Evidências da interface

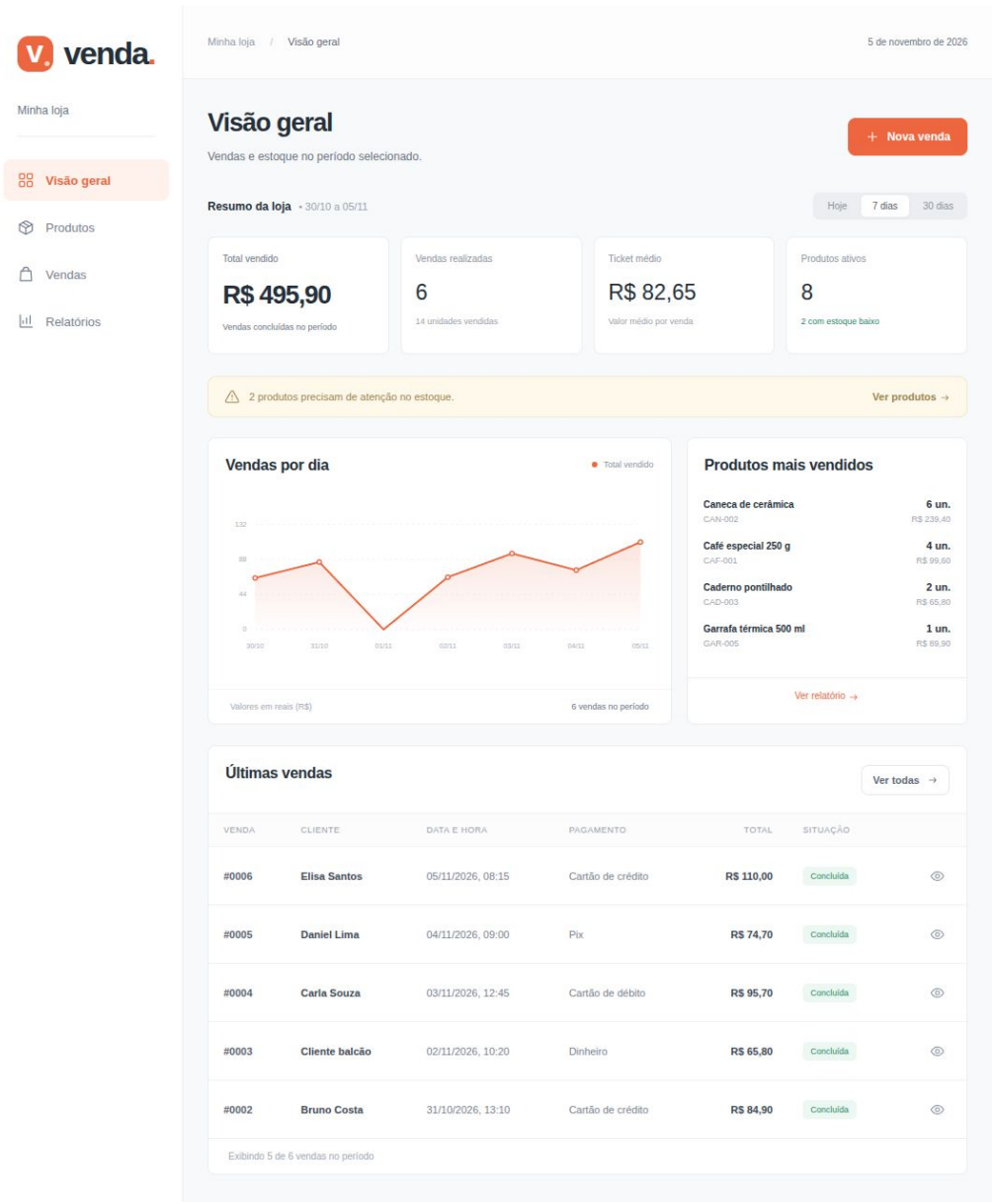


Figura 2 — Visão geral com valores consultados no banco de demonstração.

Fonte: captura da aplicação. As evidências de testes estão em artifacts/.

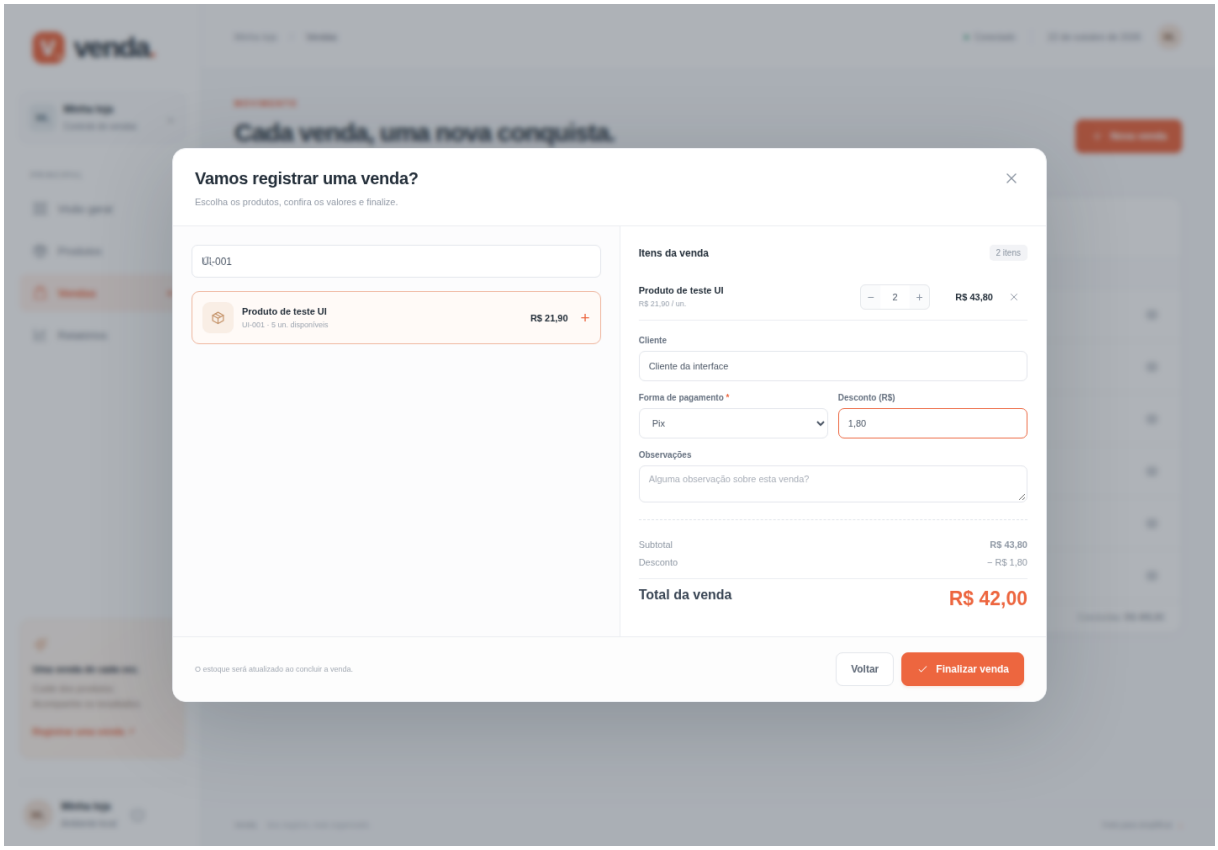


Figura 3 — Registro de venda com produtos, quantidades e desconto.

Fonte: captura da aplicação. As evidências de testes estão em artifacts/.

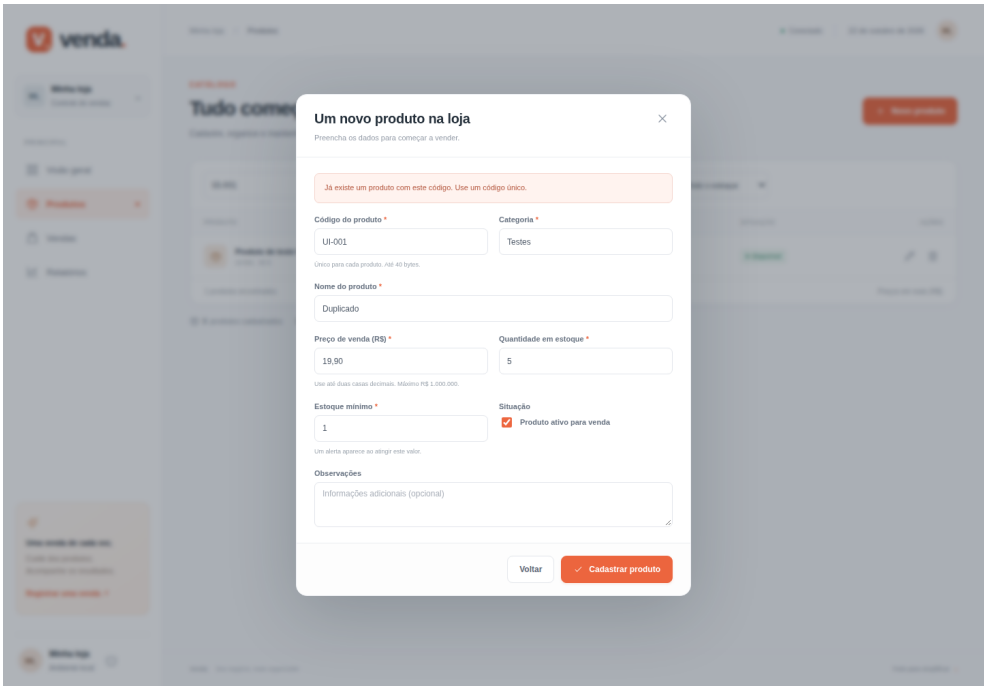


Figura 4 — Mensagem de código de produto duplicado.

Fonte: captura da aplicação. As evidências de testes estão em artifacts/.

6 MANUAL DE UTILIZAÇÃO

6.1 Compilar e executar

O projeto requer Linux ou WSL, compilador C11 (GCC/Clang), GNU Make e navegador atual. O SQLite e o cJSON já acompanham os fontes. Node e Python não são exigidos para usar o sistema; Python 3 é necessário apenas para os testes e as ferramentas de entrega.

```
./iniciar.sh
# Abrir http://127.0.0.1:8080

# Alternativa manual:
make
./build/vendas

# Exemplos opcionais em um banco vazio:
./iniciar.sh --demo

# Outra porta e outro arquivo:
./build/vendas --port 8081 --db data/minha-loja.db
```

O script iniciar.sh muda para a pasta do projeto antes de compilar. O banco padrão é data/vendas.db e é criado automaticamente. O servidor pode ser encerrado por Ctrl+C no terminal ou por docker compose stop quando executado em contêiner. Fechar apenas o navegador mantém o programa em execução. Ao iniciar outra vez com o mesmo arquivo, os dados são recuperados.

6.2 Operação diária

Produtos: abrir o catálogo, selecionar Novo produto, preencher código, nome, categoria, preço e estoques e confirmar. Os ícones de lápis e lixeira permitem editar e excluir. Buscar por nome/código/categoria ou aplicar os filtros reduz a lista. Se houver vendas associadas, a exclusão é recusada; desmarcar Produto ativo impede novas vendas sem apagar o histórico.

Venda: clicar em Nova venda, localizar os produtos, adicionar as quantidades, informar cliente opcional, pagamento e desconto e selecionar Finalizar venda. O programa apresenta o total e o comprovante interno. Uma venda sem estoque suficiente ou com desconto excessivo é recusada. O catálogo é atualizado automaticamente.

Histórico: acessar Vendas, pesquisar o cliente ou número e aplicar um intervalo de datas. O botão de visualização abre os itens e valores históricos. Cancelar venda exige confirmação, devolve o estoque e preserva o registro com situação Cancelada. A impressão pode ser salva como PDF pelo navegador.

Relatórios: escolher as datas inicial e final. Conferir total vendido, quantidade de vendas, ticket médio, descontos, produtos e pagamentos. Exportar CSV gera uma planilha separada por ponto e vírgula; Imprimir permite impressão ou gravação em PDF. A receita por produto é bruta; os descontos são calculados por venda. Somente vendas concluídas entram nos totais.

6.3 Testes, cópia de segurança e documentos

```
make test
make sanitize
make package

# Ferramentas opcionais, somente para regerar documentos/UI:
python3 scripts/setup-tools.py
node tests/test_ui.cjs
PYTHONPATH=.tools/python python3 scripts/generate_docs.py
```

Os testes usam bancos separados e não alteram o arquivo de trabalho. Para uma cópia de segurança manual, encerrar o servidor antes de copiar data/vendas.db; assim, as transações em WAL são finalizadas. O arquivo data/exemplo.db acompanha a entrega com oito produtos e seis vendas fictícias. make clean remove somente objetos e executável, preservando os bancos.

7 CONCLUSÃO

A solução integra interface gráfica, servidor em C e banco persistido em arquivo, atendendo ao cadastro de produtos, registro e cálculo de vendas e relatório do total vendido. A organização em módulos e as estruturas com identificadores tornam explícita a aplicação dos conteúdos exigidos. As transações protegem os estoques, e os valores em centavos evitam erros de arredondamento nos cálculos do backend.

Os testes demonstraram os fluxos descritos, a recuperação após reinício e o tratamento de entradas inválidas. As principais decisões foram adotar um único servidor local para simplificar a execução e usar bibliotecas incorporadas para não exigir instalação de banco ou pacotes de frontend. O uso de HTTP em C e a coordenação transacional dos itens são pontos que exigem estudo e domínio por todos os integrantes.

O escopo é acadêmico e local, com atendimento sequencial de requisições, sem autenticação, emissão fiscal, operação remota multiusuário ou cálculo de lucro. Melhorias futuras podem incluir essas funcionalidades, desde que sejam planejadas e testadas. O relatório adota formatação acadêmica base, com papel A4, texto Times 12, entrelinhas 1,5 e margens esquerda/superior de 3 cm e direita/inferior de 2 cm. O modelo institucional específico não foi fornecido e deve ser conferido antes da submissão.

8 REFERÊNCIAS

PROJETO PRÁTICO — DESENVOLVIMENTO DE SOFTWARE EM C. Disciplina Programação de Software Básico em C. Documento de enunciado fornecido na atividade, 4 p., 2026. Arquivo: Projeto_Pratico_C_4_Pontos.pdf.

SQLITE. SQLite amalgamation, versão 3.50.4. Código-fonte e interface C. Disponível em: https://www.sqlite.org/2025/sqlite-amalgamation-3500400.zip. Cópia incorporada em vendor/sqlite; declaração de domínio público nos arquivos.

GAMBLE, Dave et al. cJSON, versão 1.7.19. Código-fonte e licença MIT. Disponível em: https://github.com/DaveGamble/cJSON/tree/v1.7.19. Cópia incorporada em vendor/cjson.

MICROSOFT. Playwright Core. Ferramenta opcional de automação do navegador. Disponível em: https://github.com/microsoft/playwright. Usada nos testes documentados.

REPORTLAB. ReportLab Toolkit. Ferramenta opcional para geração dos documentos PDF. Disponível em: https://www.reportlab.com/.

CANNY, Steve et al. python-pptx. Ferramenta opcional para geração da apresentação editável. Disponível em: https://github.com/scanny/python-pptx.

OPENAI. Codex. Assistência à elaboração do código, dos testes e dos materiais desta entrega. A equipe é responsável por revisar, adaptar e compreender integralmente o projeto.

9 APÊNDICE - ROTEIRO DE APRESENTAÇÃO

A identificação da equipe consta na capa. A apresentação tem 15 minutos de exposição e 5 minutos para perguntas. A distribuição abaixo organiza os temas; os responsáveis por cada etapa estão no roteiro de apoio.

Etapa	Conteúdo	Tempo
1	Problema, objetivos e requisitos.	2 min
2	Arquitetura, produtos e estruturas de dados.	2 min
3	Regras da venda, ponteiros e memória dinâmica.	3 min
4	Persistência e recuperação após reiniciar.	2 min
5	Testes e demonstração de cadastro, venda e cancelamento.	3 min
6	Relatórios, resultados e encerramento.	3 min

Na demonstração, cadastrar um produto, rejeitar um código repetido, concluir uma venda com desconto, conferir o estoque e cancelar a venda. Reiniciar o servidor e verificar a recuperação dos registros. Todos devem compreender as decisões técnicas e responder às perguntas.

Materiais de apoio: docs/apresentacao.pdf e docs/roteiro-apresentacao.txt. A relação entre os requisitos e a implementação está em docs/requisitos.md.